

Orchestrating Mayhem

Functional

Chaos Engineering

Szymon Mentel

szymon.mentel@erlang-solutions.com

[@szymonmetel](https://twitter.com/szymonmetel)

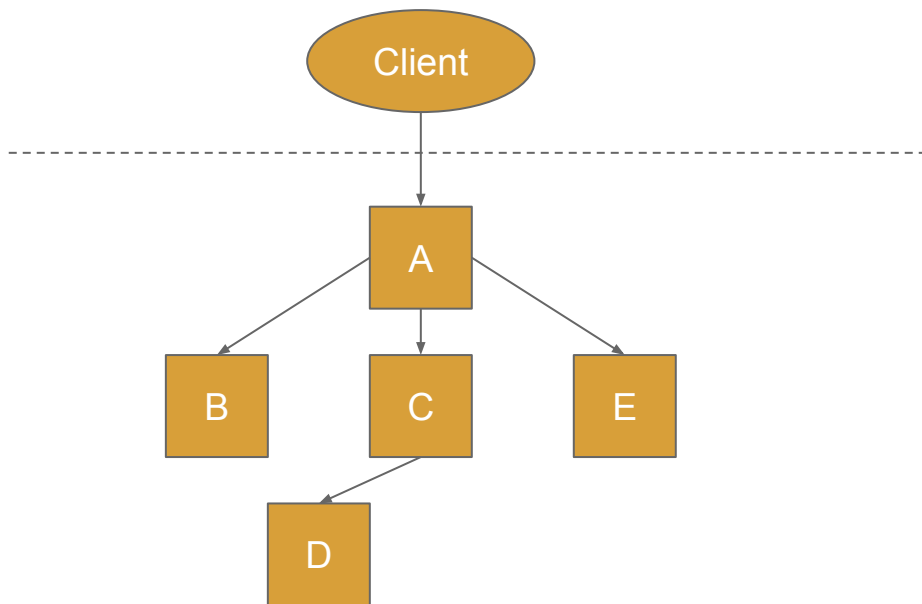
github.com/mentels



1.

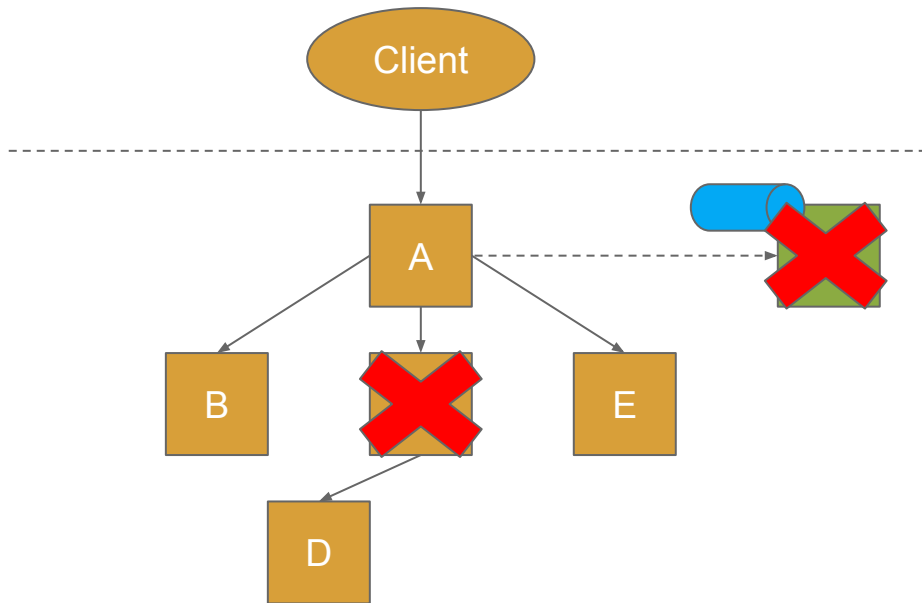
Chaos:
WHAT
WHY
HOW

WHAT is Chaos: complex systems



WHAT is Chaos: complex systems

Interactions compounded with **real-world** events
may lead to unpredictable outcomes

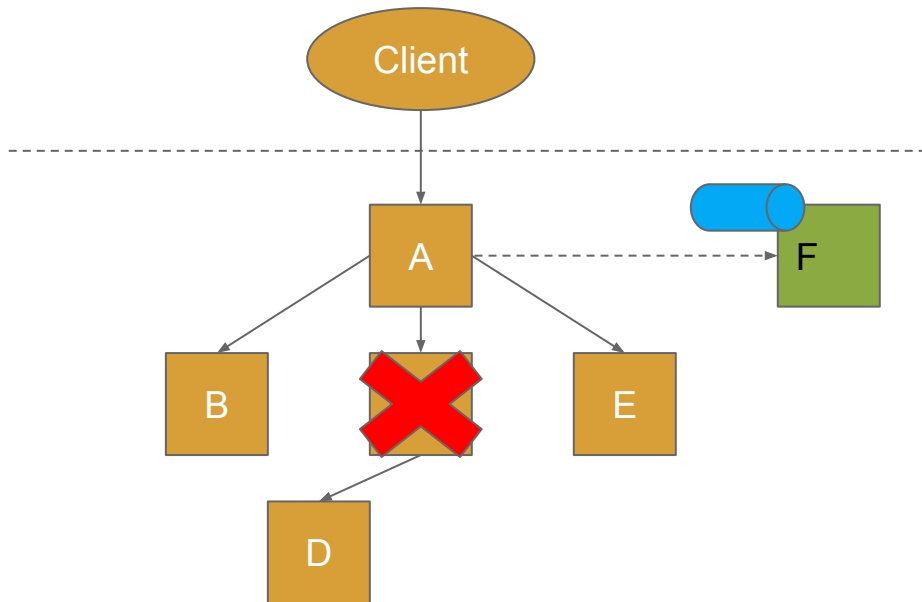


“

Chaos Engineering is the discipline of experimenting on a distributed system in order to build confidence in the system's capability to withstand turbulent conditions in production.

~ <http://principlesofchaos.org/>

Chaos Engineering: experimenting



WHY

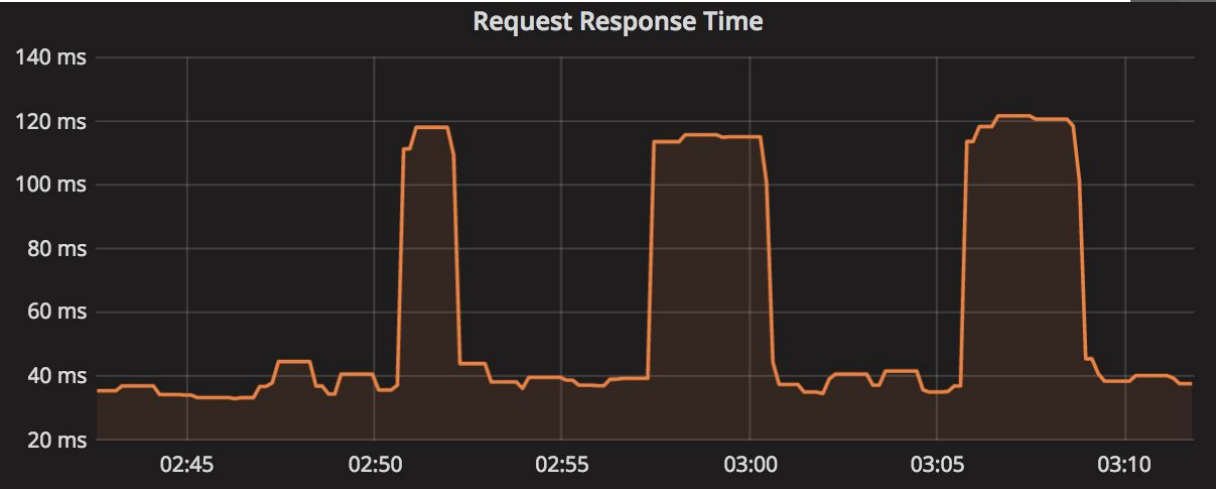
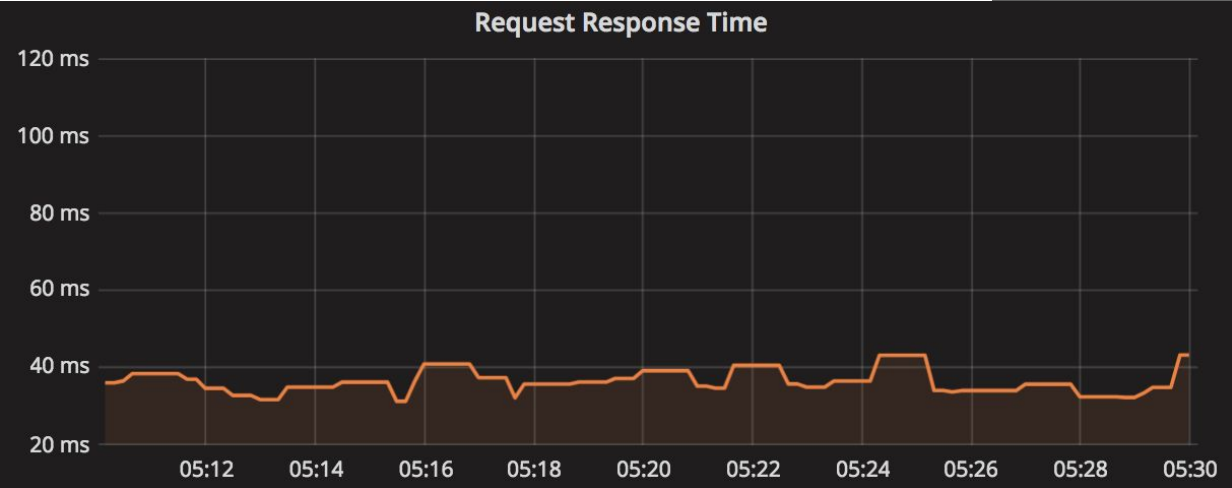
What is the rationale for Chaos Engineering?

Trust

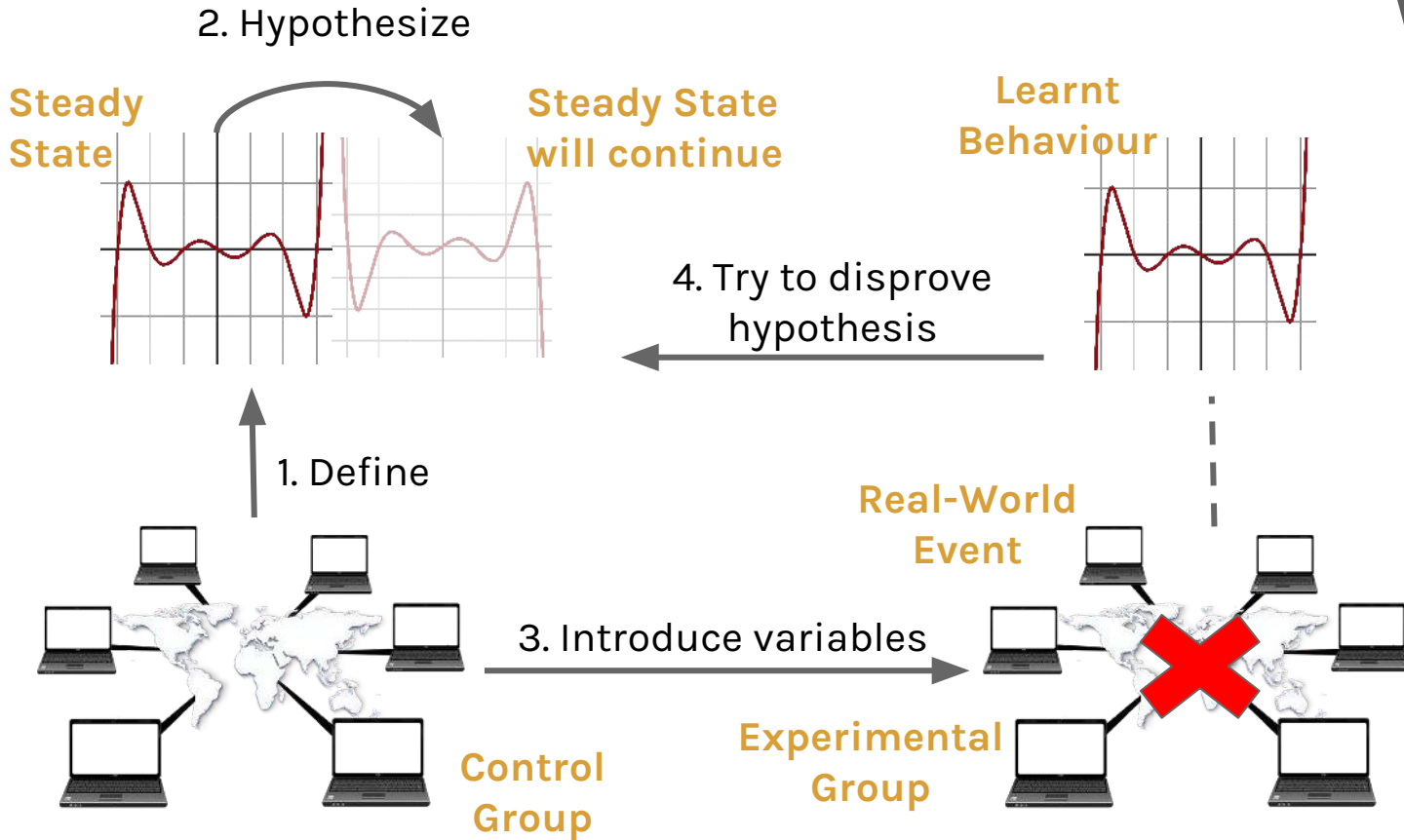
Being
Proactive

Cost
effectiveness

HOW: Steady State

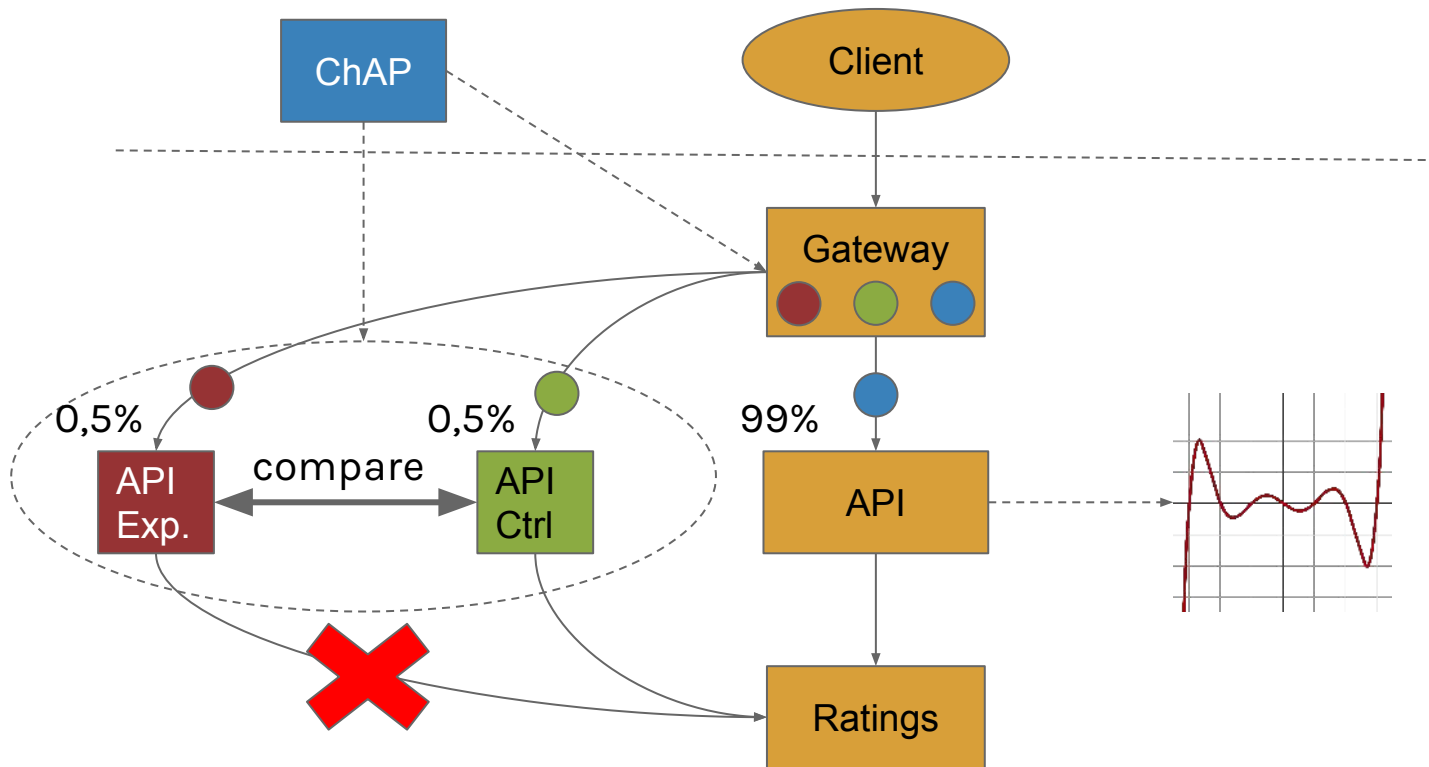


HOW: 4 steps



HOW: Chaos Engineering at Netflix

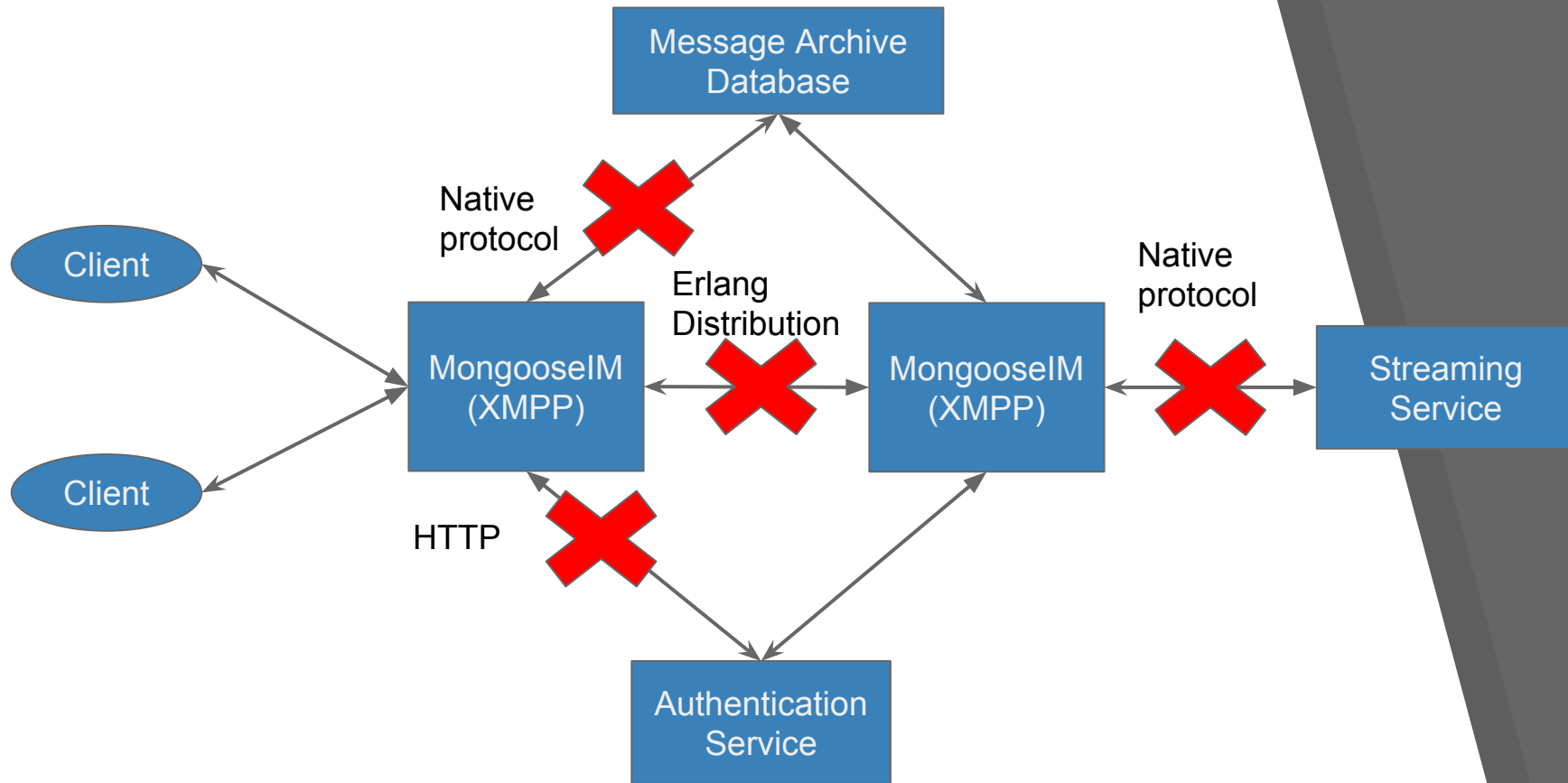
How **API** handles failure of **Rating** service?



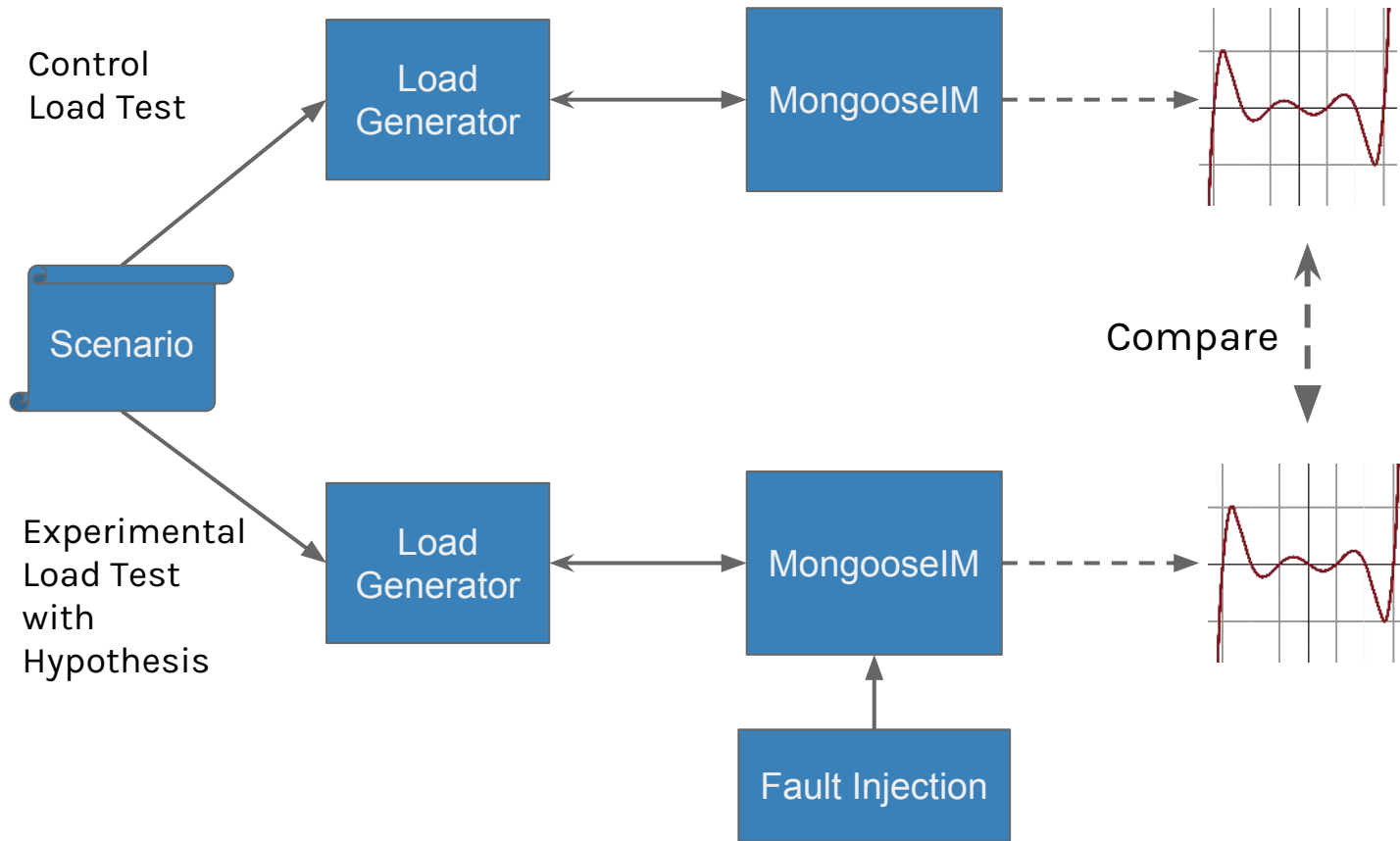
2.

Applying Chaos Engineering Principles

Applying ChE: Injection Points

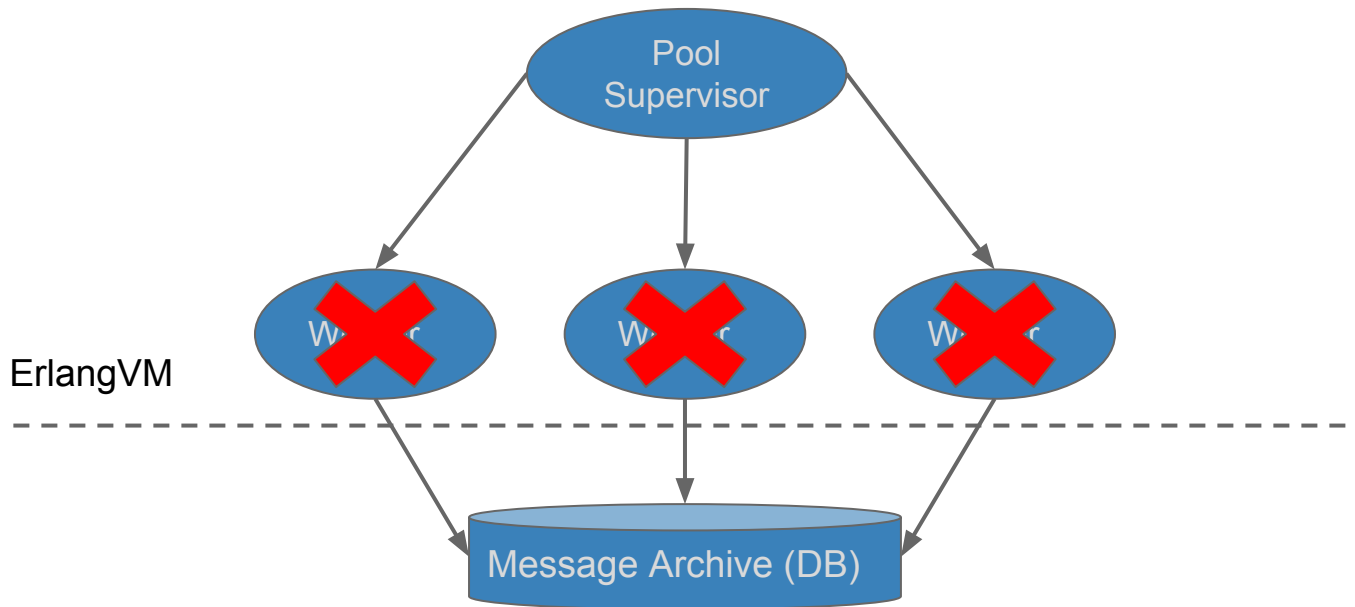


Applying ChE: Comparing



Hypothesis 1: Database Failure

Failure to write to the database won't disrupt the service



Hypothesis 1: Database Failure

```
# setup
kill_pool = fn f ->
  workers = Supervisor.which_children(PoolSup)
  Enum.each(workers,
    fn pid -> Process.exit(pid, :chaos) end)
  Process.sleep(10)
  f.(f)
end
# run
killer = spawn(fn -> kill_pool.(kill_pool) end)
# stop
Process.exit(killer, :kill)
```

Hypothesis 1: Database Failure

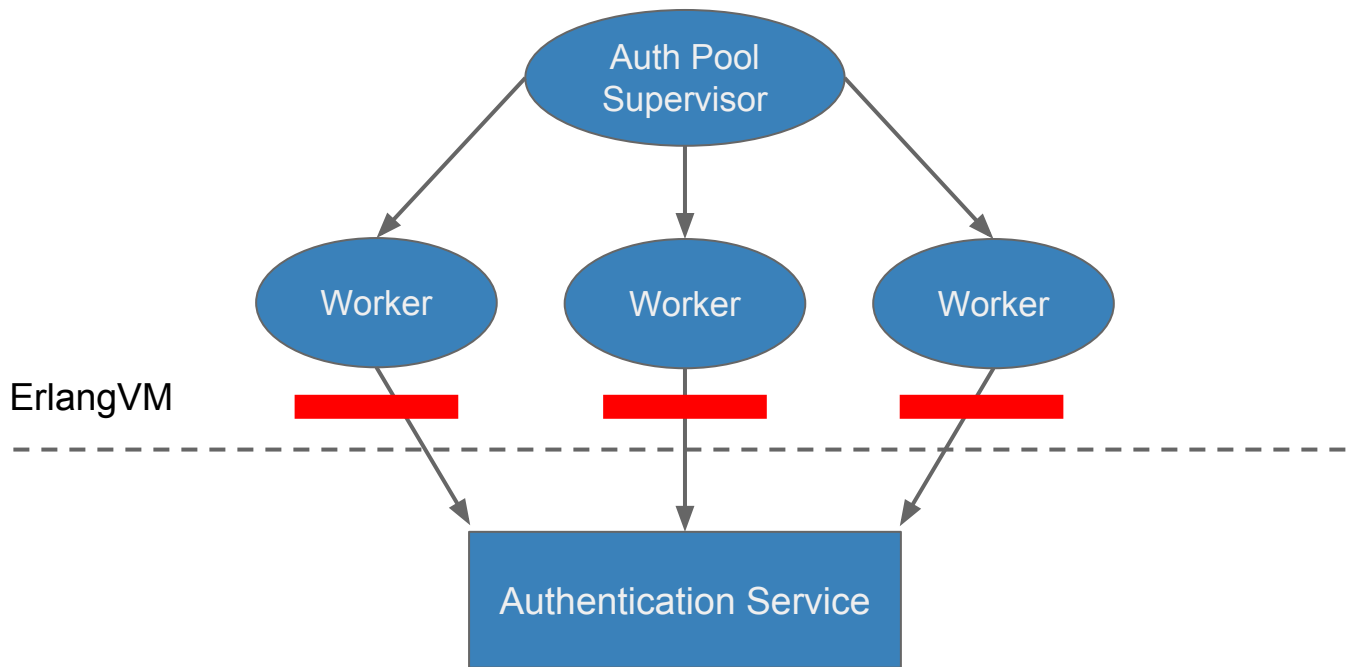
Chaos Started

Chaos Stopped



Hypothesis 2: Slow Network

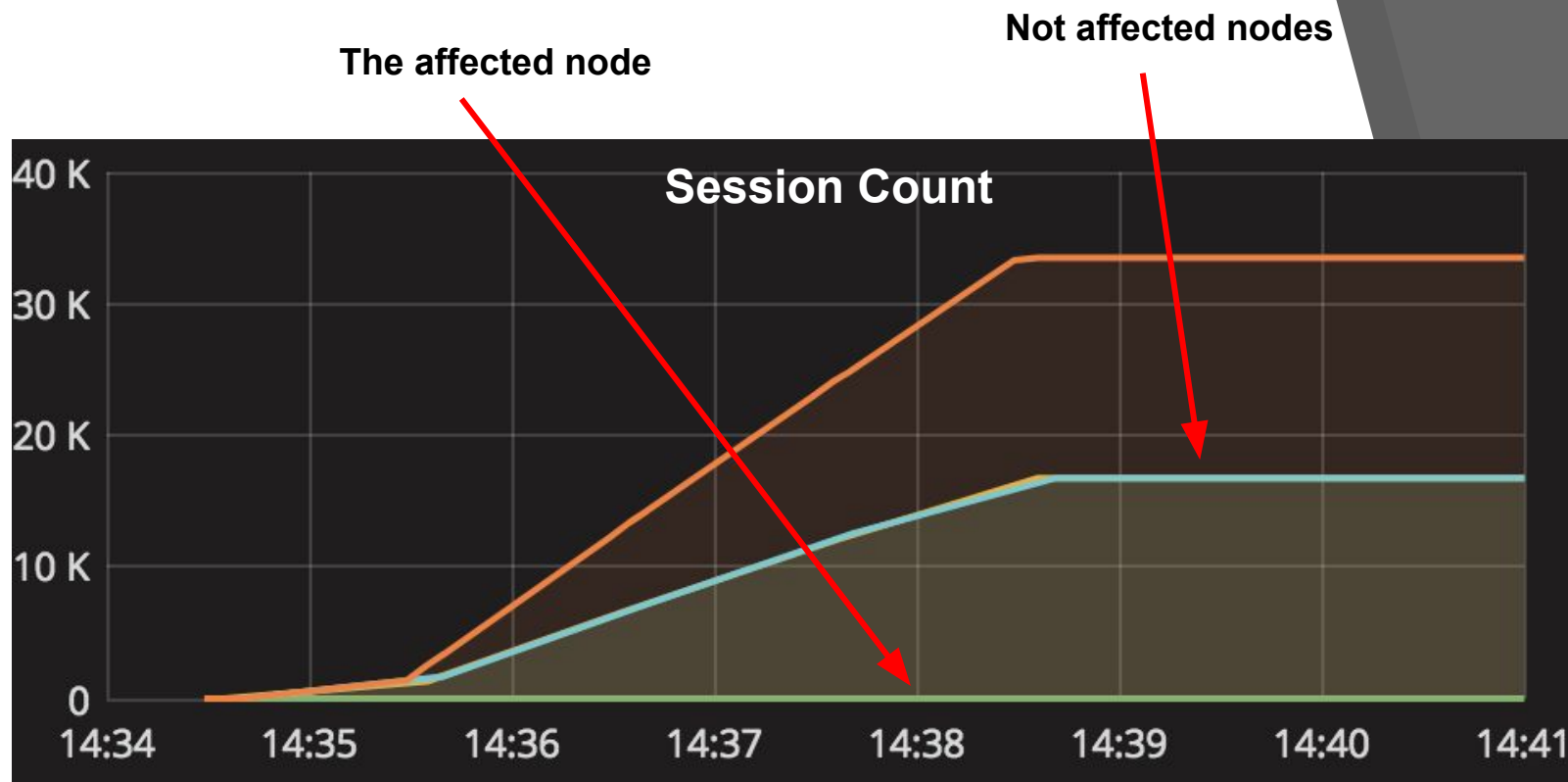
Delay on the connection to the Authentication Service won't prevent users from logging in



Hypothesis 2: Slow Network

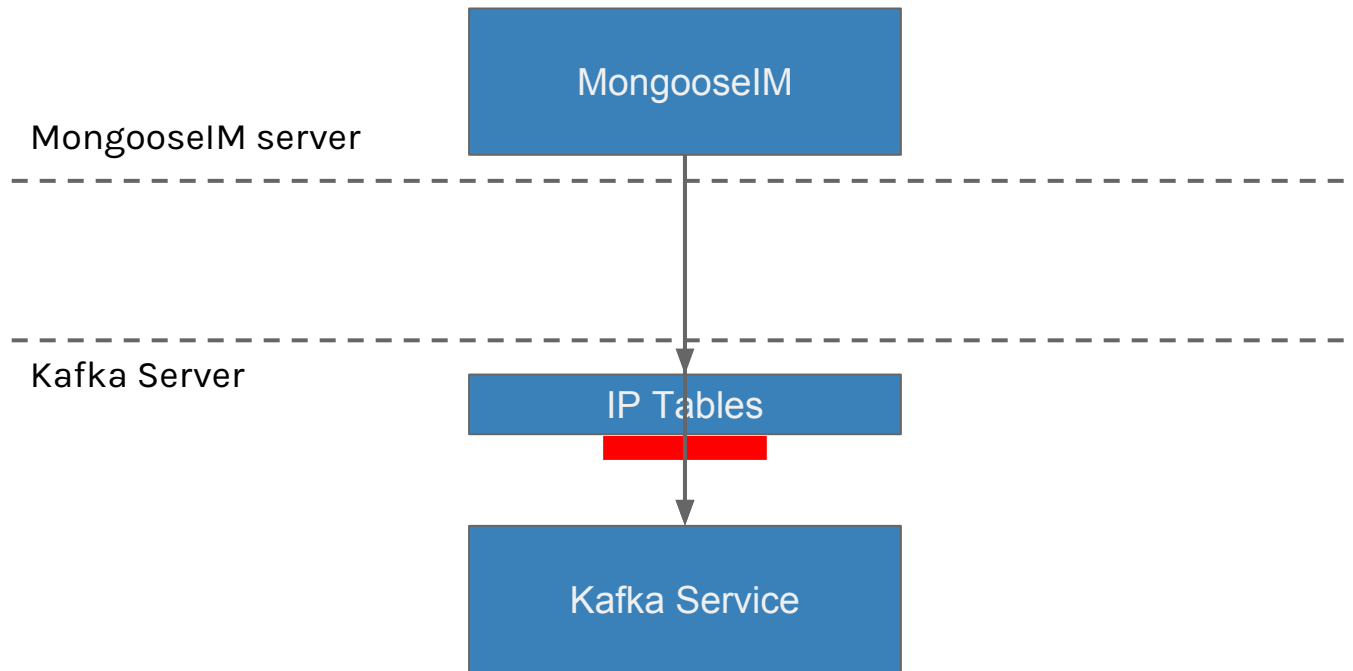
```
# setup
delay = 100
delayed_auth = fn user, pass ->
    Process.sleep(delay)
    AuthService.authenticate(user, pass)
end
:ok = :meck.new(AuthService, [:passthrough])
# run
:ok = :meck.expect(AuthService, :authenticate,
    fn user, pass -> delayed_auth(user, pass) end)
# stop
:ok = :meck.unload(AuthService)
```

Hypothesis 2: Slow Network



Hypothesis 3: Network Glitch

Network glitch on the connection to Kafka won't cause any data loss



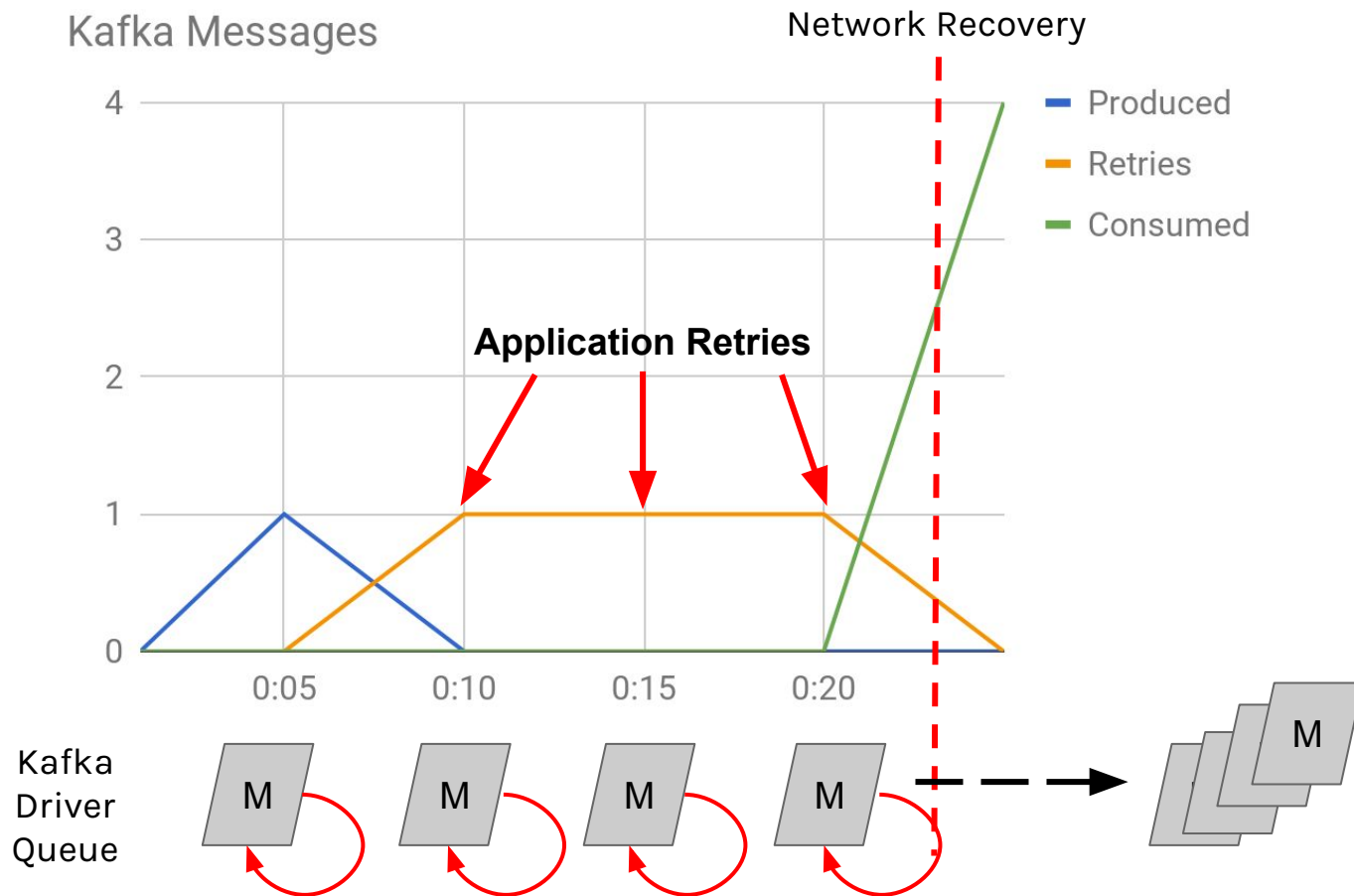
Hypothesis 3: Network Glitch

```
# setup
cmd = ["INPUT -m state --state NEW,ESTABLISHED,RELATED -p
tcp --dport 9092 -s #{mim_addr} -j DROP"]
enable_cmd = "-A" <> cmd
disable_cmd = "-D" <> cmd

# run
{_, 0} = System.cmd("iptables", enable_cmd)

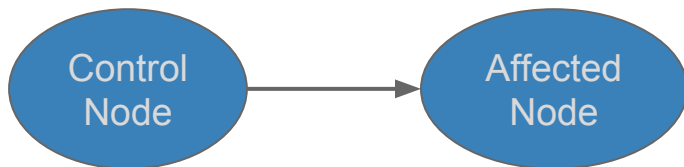
# stop
{_, 0} = System.cmd("iptables", disable_cmd)
```

Hypothesis 3: Network Glitch



Fault Injection: no recompilation

Direct or Remote



- ▶ RPC

```
:rpc.call(:aff_node@localhost, PoolSup, :which_children, [])
```

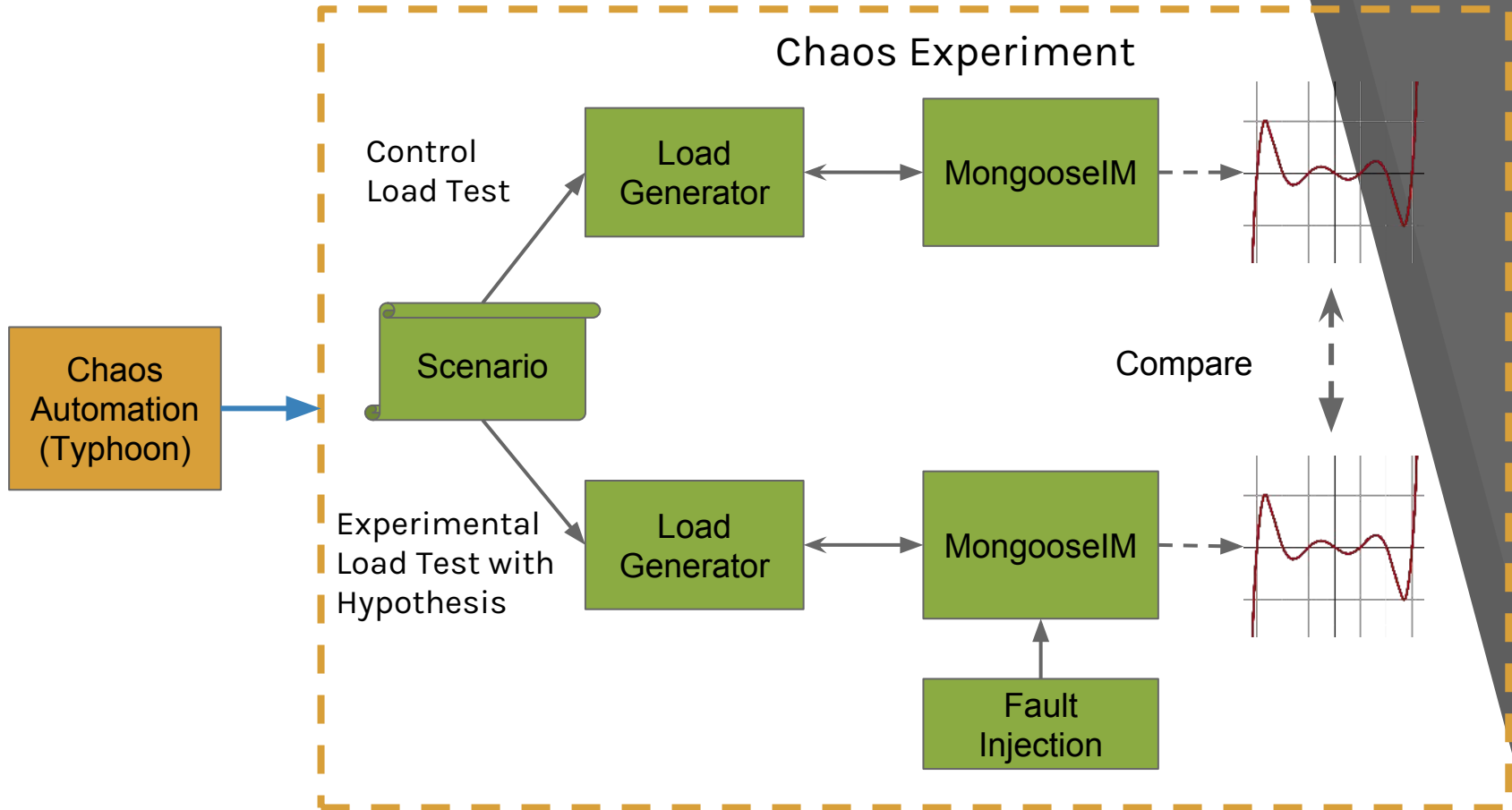
- ▶ Remote processes

```
Node.spawn(:aff_node, fn -> ... end)
```

3.

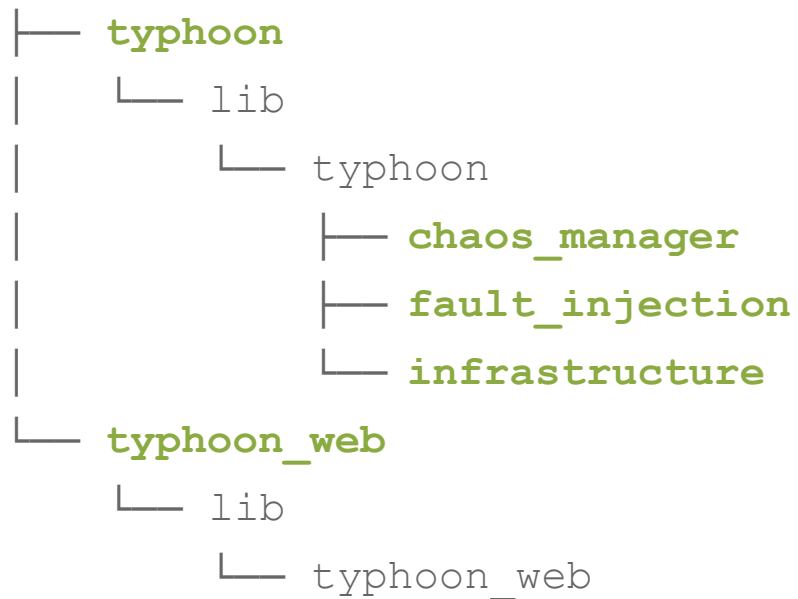
MongooselM
Chaos
Engineering
Automation

Automation of Chaos Engineering



Typhoon: Elixir Application

apps



Typhoon: Elixir Application

Infrastructure

```
%TestSetup{}  
%TestTopology{}
```

Fault
Injection

```
Fault Protocol  
%MyFault{}
```

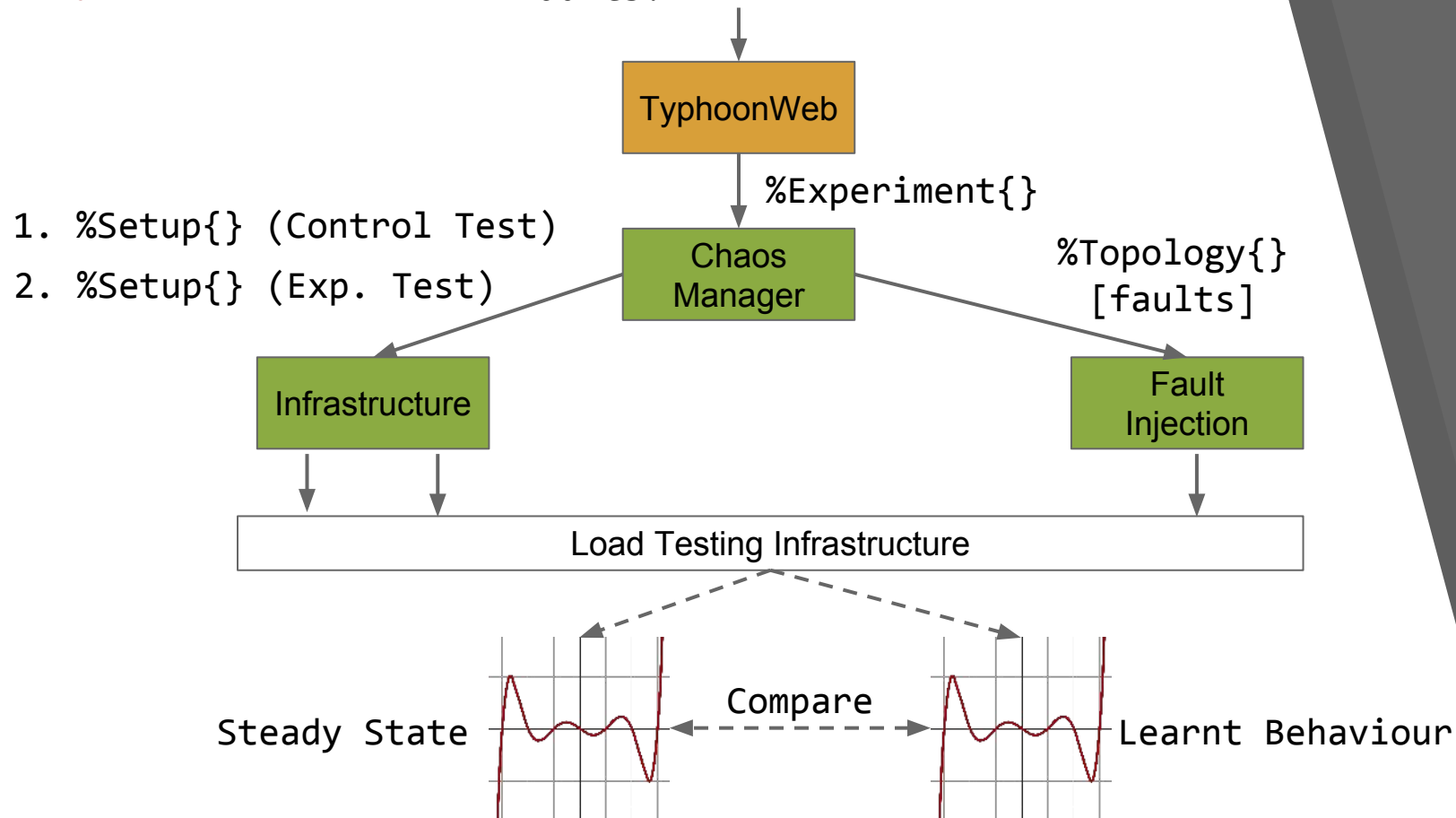
Chaos
Manager

```
%Experiment{  
  :control_test_id,  
  :experimental_test_id,  
  :setup,  
  :faults,  
  :faults_offsets,  
}
```

Typhoon: Elixir Application

Load Test Config: ...

Faults: ...



Typhoon: Fault Injection

```
defprotocol FaultInjection.Fault do
  @doc "Applies the fault to the load test run by `test_id`"
  @spec apply(struct(), test_id()) :: :ok | {:error, term()}
  def apply(fault, test_id)
end
```

```
defimpl FaultInjection.Fault, for: MyFault do
  def apply(fault, test_id), do: send self, %{fault: MyFault}
end
```

Typhoon: Fault Injection

```
defmodule FaultInjection.Fault.MyFault do
  embedded_schema do
    field(:param1, :integer)
    field(:param2, :string)
  end

  def changeset(struct, attrs) do
    struct
    |> cast(attrs, [:param1, :param2])
    |> validate_required([:param1, :param2])
  end

  defimpl FaultInjection.Fault do
    def apply(fault, test_id), do: send self, %{fault: MyFault}
  end
end
```

CHAOS ENGINEERING

is for everyone - go and explore it

APPLY

it to your system using the most basic techniques available

AUTOMATE

if it works for you add it to your continuous integration pipeline

THANK YOU!

szymon.mentel@erlang-solutions.com

@szymonmetel

github.com/mentels

medium.com/@mentels